GRUPO 4 · REQUERIMIENTOS FUNCIONALES

Registro de Usuario / Onboarding

Módulo: Alta de nuevo cliente (KYC básico)

Curso de Automatización de Pruebas de Software
Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 4
Versión	1.0
Fecha	22 de agosto de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST) y aiquaa-sandbox-web (front)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

> Documento derivado del código fuente de aiquaa-sandbox-api y aiquaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre el CRUD completo de clientes (alta, listado, consulta, reemplazo y baja) y la actualización del estado de verificación de identidad (KYC). Es el módulo que da origen a todos los demás: el `usuarioId` que devuelve el alta es la entrada de cuentas, tarjetas, facturas, órdenes, reservas, notificaciones y roles.

Fuera de alcance: Carga de documentos respaldatorios, validación contra fuentes externas, puntaje de riesgo, y flujo de aprobación con revisor: el estado de KYC se fija directamente por API.

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con su propia API key y automatiza escenarios BDD sobre el módulo de su grupo.
Usuario de negocio (fila de `usuarios`)	Identidad de dominio sobre la que operan los endpoints: es dueño de cuentas, facturas, tarjetas, órdenes, reservas, notificaciones y roles.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo bajo el rol de base de datos `qa_api` (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz que consume la API a través del proxy `/api/proxy/{path}`; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
GET /api/v1/usuarios	RF-G4-04	Lista los clientes activos del sandbox.
POST /api/v1/usuarios	RF-G4-01	Da de alta un cliente nuevo con KYC pendiente.
GET /api/v1/usuarios/{id}	RF-G4-02	Devuelve la ficha de un cliente.
PUT /api/v1/usuarios/{id}	RF-G4-05	Reemplaza los datos de identificación de un cliente.
DELETE /api/v1/usuarios/{id}	RF-G4-06	Da de baja (soft-delete) un cliente.
PATCH /api/v1/usuarios/{id}/kyc	RF-G4-03	Actualiza el estado de verificación de identidad.

2. Precondiciones y acceso

- Toda request a la API debe enviar el header `x-api-key` con una clave existente y con `active = true` en la tabla `public.api_keys`; sin header o con clave inválida/inactiva la respuesta es `401 UNAUTHORIZED` con el mensaje genérico "Invalid or inactive API key.".
- Cada API key admite un máximo de **200 requests por minuto** (ventana deslizante). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset`.
- Los datos de práctica viven en el schema aislado `qa_training` y se cargan con `scripts/seed-data.sql`; la data es determinística, por lo que un escenario puede apoyarse en los registros sembrados.

- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE` en Postgres, y todo endpoint `DELETE` de la API hace baja lógica (`activo = false`), devolviendo `204 No Content` sin cuerpo.
- Casi todas las tablas de negocio (`sesiones`, `transferencias`, `facturas`, `tarjetas`, `notificaciones`, `ordenes`, `reservas`, `movimientos`, `roles`, además de `usuarios` y `cuentas` que ya la tenían) tienen columna `activo`/`activa`; todo `GET`, `PUT` y `DELETE` de esa tabla filtra u opera solo sobre filas con `activo = true` — una fila dada de baja deja de ser visible y de poder reemplazarse, pero no se borra.
- En la aplicación web el acceso tiene dos capas: capa 1 la API key (pantalla `/login`, se valida contra `GET /api/v1/roles` antes de guardarse) y capa 2 el usuario de negocio (pantalla `/auth/login`). Con `NEXT_PUBLIC_DEMO_MODE=true` la capa 1 se omite y el proxy inyecta una key demo del servidor.
- Las rutas `/usuarios/*` de la web son la excepción al guard de capa 2: se pueden usar solo con la API key, porque son el punto de partida para obtener un `usuarioId`.
- El email y el número de documento son únicos en todo el sandbox: cada ejecución de un escenario de alta debe generar valores nuevos (por ejemplo, con un sufijo aleatorio o la marca de tiempo), o el segundo intento fallará.
- El alta es la única operación del sandbox que puede ejecutarse desde la web con la sola API key, sin haber iniciado sesión como usuario de negocio.

3. Modelo de datos del módulo

usuarios

Ficha del cliente. Este módulo la crea, la consulta y actualiza su estado de KYC.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`nombre`	`text`, obligatorio
`email`	`text`, obligatorio y único en todo el sandbox
`activo`	`boolean`, obligatorio, por defecto `true`
`documento_tipo`	`text`, uno de `CI` / `pasaporte` / `RUC`; por defecto `CI`
`documento_numero`	`text`, obligatorio y único en todo el sandbox
`fecha_nacimiento`	`date`, opcional
`direccion`	`text`, opcional
`kyc_estado`	`text`, uno de `pendiente` / `verificado` / `rechazado`; por defecto `pendiente`
`created_at`	`timestamptz`, por defecto `now()`

> El alta no permite elegir el estado de KYC ni el estado de actividad: todo cliente nuevo nace con `kyc_estado = 'pendiente'` y `activo = true`. Como el login del Grupo 1 solo exige que el usuario esté activo, un cliente recién dado de alta puede iniciar sesión aunque su identidad no esté verificada.

4. Requerimientos funcionales

RF-G4-01 — Dar de alta un cliente

`POST /api/v1/usuarios`

Registra un cliente nuevo con sus datos de identificación mínimos. El cliente queda operativo de inmediato, con su verificación de identidad pendiente.

Entradas y validaciones

- `nombre` (obligatorio): texto no vacío.
- `email` (obligatorio): texto con formato de correo electrónico válido.
- `documentoTipo` (obligatorio): uno de `CI`, `pasaporte` o `RUC`.
- `documentoNumero` (obligatorio): texto no vacío.
- `fechaNacimiento` (opcional): fecha; si se omite se guarda vacía.
- `direccion` (opcional): texto libre; si se omite se guarda vacía.

Reglas de negocio

- El email y el número de documento deben ser únicos: un valor repetido es rechazado por la base de datos con `409 CONFLICT`.
- El cliente nuevo queda siempre con `kyc_estado = 'pendiente'` y `activo = true`; ninguno de los dos puede enviarse en el alta.
- El tipo de documento se limita a las tres opciones válidas, tanto en la entrada como en la base.
- La fecha de nacimiento no se valida contra ninguna edad mínima: se acepta cualquier fecha, incluso futura, mientras sea una fecha reconocible.
- No se valida coherencia entre el tipo y el formato del número de documento.

Respuesta esperada

- `201 Created` con `data` conteniendo la ficha completa del cliente creado, incluido su `id` y su `kyc_estado` en `pendiente`.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta un campo obligatorio, el email no tiene formato válido o el tipo de documento no está permitido.
`CONFLICT`	409	El email o el número de documento ya pertenecen a otro cliente.
`EXECUTION_ERROR`	400	La fecha de nacimiento no es una fecha válida.

Fuente: `app/api/v1/usuarios/route.ts`

RF-G4-02 — Consultar la ficha de un cliente

`GET /api/v1/usuarios/{id}`

Devuelve la ficha completa de un cliente, incluidos sus datos de identificación y su estado de verificación.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Devuelve el cliente cuyo identificador coincide exactamente, siempre que siga `activo = true` (dado de baja con RF-G4-06 responde 404).
- Si no existe o ya está dado de baja, responde 404 con el mensaje "Usuario no encontrado.".
- La ficha se devuelve completa, sin enmascarar el número de documento.

Respuesta esperada

- `200 OK` con `data` conteniendo la ficha completa del cliente.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un cliente con ese identificador, o ya está dado de baja.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/usuarios/[id]/route.ts`

RF-G4-03 — Actualizar el estado de verificación (KYC)

`PATCH /api/v1/usuarios/{id}/kyc`

Fija el estado de verificación de identidad de un cliente, para reflejar el resultado de la revisión de sus datos.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `kycEstado` (obligatorio, en el cuerpo): uno de `pendiente`, `verificado` o `rechazado`.

Reglas de negocio

- El estado enviado se escribe tal cual, sin verificar el estado anterior: **cualquier transición está permitida**, incluidas `verificado` → `pendiente` y `rechazado` → `verificado`.
- Fijar el mismo estado que ya tenía el cliente es aceptado y devuelve la ficha sin cambios visibles.
- El cambio de estado no altera `activo`: un cliente con KYC rechazado sigue pudiendo iniciar sesión y operar.
- Si el cliente no existe, la operación responde 404 y no modifica nada.

Respuesta esperada

- `200 OK` con `data` conteniendo la ficha completa ya actualizada.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un cliente con ese identificador (mensaje: "Usuario no encontrado.").
`VALIDATION_ERROR`	400	`kycEstado` ausente o con un valor fuera de la lista permitida.

Fuente: `app/api/v1/usuarios/[id]/kyc/route.ts`

RF-G4-04 — Listar clientes

`GET /api/v1/usuarios`

Devuelve la lista de clientes activos del sandbox.

Entradas y validaciones

- No recibe parámetros de filtro.

Reglas de negocio

- Devuelve únicamente clientes con `activo = true`, ordenados por identificador ascendente, limitados a 100 registros.
- No admite filtro por `usuarioId`, `email`, `kycEstado` ni ningún otro campo: es un listado plano.

Respuesta esperada

- `200 OK` con `data` como arreglo de fichas de cliente.

Errores

Código	HTTP	Cuándo ocurre
`UNAUTHORIZED`	401	Falta la API key o es inválida.

Fuente: `app/api/v1/usuarios/route.ts`

RF-G4-05 — Reemplazar la ficha de un cliente

`PUT /api/v1/usuarios/{id}`

Reemplaza los datos de identificación de un cliente existente. El estado de KYC y el estado de actividad quedan fuera de este reemplazo.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `nombre` (obligatorio): texto no vacío.
- `email` (obligatorio): texto con formato de correo electrónico válido.
- `documentoTipo` (obligatorio): uno de `CI`, `pasaporte`, `RUC`.
- `documentoNumero` (obligatorio): texto no vacío.
- `fechaNacimiento` (opcional): fecha.
- `direccion` (opcional): texto libre.

Reglas de negocio

- Solo puede reemplazar un cliente con `activo = true`; uno dado de baja responde 404.
- `kycEstado` no forma parte del cuerpo: sigue gobernado exclusivamente por `PATCH .../kyc` (RF-G4-03).
- `activo` tampoco forma parte del cuerpo: darlo de baja sigue siendo exclusivo de `DELETE` (RF-G4-06).
- Reemplazar el `email` o el `documentoNumero` por uno usado en otro cliente es rechazado por unicidad.

Respuesta esperada

- `200 OK` con `data` conteniendo la ficha ya actualizada.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un cliente vigente con ese identificador.
`VALIDATION_ERROR`	400	Falta un campo obligatorio o tiene un formato inválido.
`CONFLICT`	409	El nuevo `email` o `documentoNumero` ya pertenecen a otro cliente.

Fuente: `app/api/v1/usuarios/[id]/route.ts`

RF-G4-06 — Dar de baja un cliente

`DELETE /api/v1/usuarios/{id}`

Da de baja lógica un cliente, quitándolo de los listados y consultas por id.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Marca `activo = false`; la fila permanece en la base, nunca se borra físicamente.
- No verifica si el cliente tiene cuentas, tarjetas, facturas, órdenes, reservas o roles asociados: todos esos recursos siguen existiendo y siendo consultables por su propio id después de la baja.
- Un cliente dado de baja por este endpoint ya no puede iniciar sesión: RF-G1-01 exige `activo = true`.

Respuesta esperada

- `204 No Content`, sin cuerpo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un cliente vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/usuarios/[id]/route.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo).
- **Coerción de tipos:** los identificadores y montos que llegan por query o por path se convierten de texto a número automáticamente; los campos numéricos de un cuerpo JSON deben enviarse como número, no como texto, salvo que el requerimiento indique lo contrario.
- **Cuerpo JSON malformado:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body.".
- **Errores de base de datos, mapeados por código SQLSTATE:** una violación de unicidad (`23505`, ej. un email o número de factura repetido) se traduce a `409 CONFLICT`; una violación de clave foránea o de restricción `CHECK` (`23503` / `23514`) se traduce a `400 VALIDATION_ERROR`; cualquier otro error de base de datos cae en `400 EXECUTION_ERROR`.
- `DELETE` **es siempre baja lógica:** actualiza `activo = false` (o `activa` en `cuentas`) y responde `204 No Content` **sin cuerpo** — no `200` con la fila afectada.

- **Verbo no soportado:** una ruta que no define el método HTTP usado responde `405 Method Not Allowed`; lo genera Next.js automáticamente, ninguna ruta lo arma a mano, y su cuerpo no sigue la envoltura `{ error }` del resto de la API.
- **Auditoría:** toda request, exitosa o fallida, deja un registro en `public.sql_audit_log` con la clave usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Idempotencia de las transiciones de estado:** los endpoints que fijan un estado (`bloquear`, `activar`, `confirmar`, `cancelar`, `leer`) escriben el valor destino sin verificar el estado previo, por lo que repetir la llamada devuelve el mismo resultado.
- **Las transiciones de estado anteriores al CRUD no miran `activo`:** `PATCH .../bloquear`, `.../activar`, `.../leer`, `.../confirmar`, `.../cancelar`, `.../kyc` y `POST .../pagar` no filtran por `activo = true` (ninguno de esos archivos se tocó al agregar el CRUD). Un recurso dado de baja con el nuevo `DELETE` deja de aparecer en `GET`, pero **sigue pudiendo bloquearse, confirmarse, leerse o pagarse** a través de su acción dedicada — una inconsistencia real del sandbox, no un comportamiento a asumir como intencional.

Códigos de éxito

Status	Cuándo ocurre
`200 OK`	GET/PUT/PATCH exitoso; también un POST de upsert que reactivó una fila existente en vez de crearla.
`201 Created`	POST que crea una fila nueva.
`204 No Content`	DELETE exitoso (baja lógica) — la respuesta no lleva cuerpo.

Códigos de error

Código	HTTP	Significado
`UNAUTHORIZED`	401	Falta el header `x-api-key`, o la clave no existe o está inactiva.
`RATE_LIMITED`	429	Se superaron las 200 requests por minuto de esa API key.
`VALIDATION_ERROR`	400	La entrada no cumple el esquema de la ruta, el JSON es inválido, o Postgres rechazó una clave foránea/CHECK.
`EXECUTION_ERROR`	400	La consulta falló en la base de datos por un motivo no cubierto por los códigos anteriores.
`NOT_FOUND`	404	El recurso indicado no existe, ya está dado de baja (`activo=false`), o no está en un estado que admita la operación.
`CONFLICT`	409	Violación de restricción `UNIQUE` en Postgres (ej. email, número de factura o nombre de rol duplicado).
`INTERNAL_ERROR`	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/{path}`, que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú de navegación muestra únicamente los módulos del grupo del alumno cuando su email figura en el roster del curso; si no figura, muestra los diez módulos. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local. **La interfaz web todavía no tiene pantallas para `PUT` / `DELETE` ni para los recursos nuevos (`sesiones` , `movimientos`)**: solo cubre los flujos de creación y consulta que ya existían; esos requerimientos se prueban llamando la API directamente (Postman, `curl`, o el cliente HTTP del framework de automatización), no navegando la web.

Pantalla	Qué permite hacer
`/usuarios/new` — Alta de cliente	Formulario con nombre, email, tipo y número de documento, y los campos opcionales de fecha de nacimiento y dirección. Ejecuta RF-G4-01 y, al crearse la ficha, navega a su detalle.
`/usuarios/{id}` — Ficha del cliente	Muestra los datos del cliente y el bloque "Actualizar KYC" con el selector de estado. Ejecuta RF-G4-02 y RF-G4-03.

- Las pantallas de este módulo son accesibles con solo la API key, sin usuario de negocio en sesión: son el punto de partida para conseguir un `usuarioId` con el que operar el resto de los módulos.
- Por esa razón, el enlace "Usuarios" aparece siempre en el menú, cualquiera sea el grupo del alumno.
- Tras actualizar el KYC, la ficha en pantalla se refresca con el estado devuelto por la API.
- No hay pantalla de listado, edición ni baja de clientes: `GET /usuarios` (lista), `PUT` y `DELETE /usuarios/{id}` (RF-G4-04/05/06) solo se pueden ejercitar llamando la API directamente.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G4-01 — Dar de alta un cliente

- Al dar de alta un cliente con datos válidos y únicos, la respuesta es 201, `data.id` es un número y `data.kyc_estado` es `pendiente`.
- El cliente creado queda con `activo = true`, por lo que puede iniciar sesión con RF-G1-01 usando el email registrado.
- Al dar de alta un cliente con un email ya existente, la respuesta es 409 `CONFLICT` y no se crea ninguna ficha.
- Al dar de alta un cliente con un número de documento ya existente, la respuesta es 409 `CONFLICT`, aunque el email sea nuevo.
- Al omitir el nombre, o al enviar un email sin formato válido, la respuesta es 400 `VALIDATION_ERROR`.
- Al enviar `documentoTipo=DNI`, la respuesta es 400 `VALIDATION_ERROR` porque el valor no pertenece a la lista permitida.
- El alta sin `fechaNacimiento` ni `direccion` es aceptada y esos campos quedan vacíos en la ficha.

RF-G4-02 — Consultar la ficha de un cliente

- Dado el `id` devuelto por un alta, al consultar la ficha la respuesta es 200 y los datos coinciden con los enviados en el alta.
- Al consultar un identificador inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Usuario no encontrado.".
- Al consultar la ficha de un cliente dado de baja con RF-G4-06, la respuesta es 404, no 200.

RF-G4-03 — Actualizar el estado de verificación (KYC)

- Dado un cliente con KYC pendiente, al fijar `verificado` la respuesta es 200 y `data.kyc_estado` es `verificado`.
- Al consultar luego la ficha con RF-G4-02, el estado persiste como `verificado`.
- Al fijar `rechazado` sobre un cliente ya verificado, la respuesta es 200: el sistema no impide el retroceso.
- Un cliente con KYC `rechazado` sigue pudiendo iniciar sesión con RF-G1-01, porque el login solo mira `activo`.
- Al enviar `kycEstado=aprobado`, la respuesta es 400 `VALIDATION_ERROR` y el estado no cambia.
- Al actualizar el KYC de un cliente inexistente, la respuesta es 404 `NOT_FOUND`.

RF-G4-04 — Listar clientes

- Al listar clientes, la respuesta es 200 y `data` contiene como máximo 100 elementos.
- Un cliente recién dado de alta aparece en este listado.
- Un cliente dado de baja con RF-G4-06 deja de aparecer en este listado.

RF-G4-05 — Reemplazar la ficha de un cliente

- Dado un cliente existente, al reemplazar su `direccion` la respuesta es 200 y el cambio se refleja al consultarlo.
- El `kyc_estado` del cliente no cambia por este reemplazo.
- Al reemplazar el `email` por el de otro cliente existente, la respuesta es 409 `CONFLICT`.
- Al reemplazar un cliente inexistente o dado de baja, la respuesta es 404 `NOT_FOUND`.

RF-G4-06 — Dar de baja un cliente

- Dado un cliente vigente, al darlo de baja la respuesta es 204 sin cuerpo.
- Tras la baja, `GET /usuarios/{id}` sobre ese cliente responde 404, y el login de RF-G1-01 con su email responde 400 (usuario no encontrado o inactivo).
- Dar de baja a un cliente no elimina ni oculta sus cuentas o tarjetas existentes, consultables por su propio id.

8. Anexo normativo (BCP / SEDECO) y brechas

> Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Ley 1015/97 y sus modificatorias — debida diligencia del cliente (SEPRELAD, supervisión del BCP)	Identificación y verificación del cliente antes de habilitarlo a operar, con documentos respaldatorios.	Parcial: se registran tipo y número de documento, pero el cliente queda operativo con la verificación pendiente y sin respaldos.
Debida diligencia ampliada y perfil de riesgo del cliente	Clasificar al cliente por riesgo y restringir su operatoria mientras la verificación no esté aprobada.	No implementado: no hay perfil de riesgo, y un cliente con KYC pendiente o rechazado puede operar todos los módulos.
Ley 6534/2020 de protección de datos personales crediticios	Tratar los datos personales con finalidad determinada y limitar su exposición.	No implementado: la ficha se devuelve completa, con el número de documento sin enmascarar, a cualquier portador de una API key válida.
Ley 1334/98 de Defensa del Consumidor (SEDECO), información al contratar	El cliente debe conocer qué datos se le solicitan y con qué finalidad al darse de alta.	Parcial: el formulario indica qué campos son obligatorios y cuáles opcionales, pero no hay aviso de privacidad ni consentimiento.
Capacidad legal para contratar	Verificar la edad mínima del titular antes del alta.	No implementado: la fecha de nacimiento es opcional y no se valida contra ninguna edad mínima.

Brechas conocidas del sandbox

- El estado de KYC puede moverse en cualquier dirección, sin máquina de estados ni registro de quién lo cambió ni cuándo.
- No existe traza de auditoría específica del cambio de KYC más allá de la bitácora general de requests.
- El KYC no condiciona ninguna otra operación del sandbox: no bloquea login, transferencias, tarjetas ni pagos.
- No hay verificación de unicidad previa al alta: el duplicado se descubre recién al insertar, aunque desde la actualización de CRUD ya se traduce a `409 CONFLICT` en vez de un `400` genérico.
- No se valida coherencia entre tipo y formato del documento (por ejemplo, un RUC con formato de cédula es aceptado).
- Dar de baja un cliente (`DELETE`) no verifica ni bloquea sus cuentas, tarjetas, facturas, órdenes, reservas ni roles asociados: todos siguen existiendo y siendo consultables por su propio id.
- El reemplazo (`PUT`) no valida coherencia entre `documentoTipo` y `documentoNumero`, igual que el alta.